BR-Bench: A Benchmark Linking Continuous Integration Changes to Fault-Revealing Tests

Deepanshu*, Sanskriti Singh*, Prisha Vadhavkar*, Dr. Yoga Raja C A†
*School of Computer Science Engineering and Information Systems
Vellore Institute of Technology, Vellore, Tamil Nadu, India
{deepanshu.2023, sanskriti.singh2023, prisha.vadhavkar2023}@vitstudent.ac.in
†Assistant Professor Sr. Grade 1, Department of Information Technology
School of Computer Science Engineering and Information Systems
Vellore Institute of Technology, Vellore, Tamil Nadu, India
yogaraja.ca@vit.ac.in

***Abstract*—BR-BENCH is a dataset that links pull-request changes to the individual tests they cause to fail in continuous integration (CI). We mined GitHub Actions runs of Java and Python repositories, parsed test verdicts out of Maven, Gradle and pytest job logs, and paired each failing head run with a base run (a run of the same workflow on the branch the pull request targets). A test is fault-revealing if it fails at the head, does not fail in the base run, and does not flip between runs of one commit. The strict split holds 762 failing runs with 4,168 fault-revealing labels on 2,466 distinct tests from 37 repositories. Of 5,985 test identifiers, 5,643 (94.29%) bind to a source file in a pinned clone, and 3,820 (63.83%) bind at full confidence. As one use of the data we compare cheap proxies for the tests a change breaks; at $k = 10$, failure history recalls 39.44% of the fault-revealing test files and file co-change 4.64%. No language model takes part in the pipeline, and one offline command regenerates the reported numbers.**

***Index Terms*—continuous integration, test selection, regression testing, mining software repositories, fault revelation**

I. INTRODUCTION

BR-BENCH is a dataset of CI runs in which every failing pull-request run is linked to the tests that the change caused to fail. We call these tests fault-revealing (tests that fail at the head commit, do not fail in the matching base run, and give the same verdict in every run of one commit). The labels come from what GitHub Actions actually executed in 76 repositories, not from a static model of the code.

The underlying question is short. Given a diff, which tests will fail? At Meta the answer comes from a model trained on internal test logs, see Machalica et al. [1], and Memon et al. [2] describe how large the problem is at Google; neither body of data is public. Regression test selection takes another route. Tools such as Ekstazi [3] and STARTS [4] compute, from file or class dependencies, a safe superset of the tests a change may affect. Change impact analysis usually asks which entities are edited together, often from version history [5], [6]; that is a statement about editing, not about failure. Public CI datasets stop before the test level of a single change (Section VI).

Three things make the labels hard to obtain. GitHub Actions keeps test output only as free text, so verdicts must be parsed. A test that fails at the head may already fail at the base. And some tests are flaky. We answer the first with deterministic parsers, the second with base-run resolution, and the third with a filter on same-commit flips.

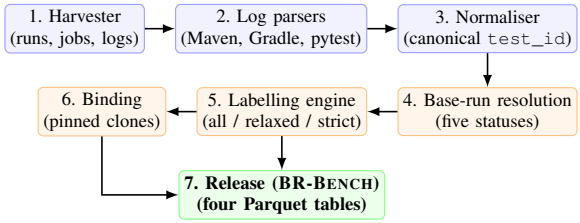


Fig. 1. The seven stages that turn GitHub Actions runs into the BR-BENCH tables.

Summary of results.

- The strict split has 762 instances, 4,168 labels and 2,466 distinct tests from 37 repositories (Section IV, Table III).
- 94.29% of test identifiers bind to a file, 63.83% at full confidence (Table IV).
- 43.88% of failed runs have no usable base run, and 0.54% of test groups flip on one commit (Section II).
- At $k = 10$, failure history recalls 39.44% of fault-revealing test files and co-change 4.64% (Section V, Fig. 2).
- The release misses the 5,000-label target and the full-confidence binding target we set ourselves (Section VII).

II. DATA SOURCE AND COLLECTION

The harvester, the parsers and the labelling engine form a pipeline of seven stages (Fig. 1). We take them in order.

Repository sampling. Candidates come from the SEART search tool for GitHub, as described by Dabić et al. [7]. Of 3,671 exported repositories, 2,646 had at least 100 GitHub Actions runs in the 90 days before export, and 2,333 of these had a workflow that runs tests rather than only linting or deployment. We drew 300 of them, balanced between Java and Python, into a sampling frame. The harvester ran from 8 to 29 August 2026 and reached 77 repositories with 175,038 runs; the instance table covers 76 of them and 165,349 runs. For each run it stored run and job metadata, check-run annotations and the raw logs, all through the GitHub REST API.

Log parsing. Deterministic line parsers read Maven Surefire and Failsafe, Gradle and pytest output and emit, for

each test, a verdict (pass, fail, error or skip) and a duration where the log prints one. Each test then gets a canonical identifier, the `test_id`. We write Java tests as `fully.qualified.Class#method` and Python tests as `path::name`. If a log names only a bare class, we recover its package from other lines of the same log where we can.

Base-run resolution. Let H be the head commit of a failed pull-request run and B the commit of the target branch. A base run is a run of the same workflow that tells us which tests already failed without the change. We search for one in a fixed order and record which case applied: `exact_green` (a successful base run whose log shows that tests ran and none failed, so the base failure set is empty), `exact` (another run on B), `ancestor` (a run on an ancestor of B at most 10 commits away), `branch_prior` (the latest run on the target branch that started before the head run) or `no_base`. A successful run whose log does not show tests running counts as no base at all, and a run with `no_base` yields no label. Of the 12,581 failed runs, 5,520 (43.88%) end as `no_base`. Among the 6,025 resolved runs with a recorded distance, the median is 0 commits, the 90th percentile 2 and the maximum 10.

Labels and splits. We write F_H for the set of tests that fail in a head run, F_B for the set that fail in its base run, and X for the same-SHA flips (tests that fail in some, but not all, runs of one workflow on one commit). The fault-revealing set is

$$T_{\text{reveal}} = (F_H \setminus F_B) \setminus X. \tag{1}$$

The labelling engine emits three nested splits. The `all` split holds every head failure in a run with a usable base and parsed head output; `relaxed` removes F_B; `strict` also removes X. A test is flaky if its verdict changes while the code does not, see Bell et al. [8], and same-SHA flips are the form of flakiness our data can observe. Flips are rare. They occur in 62 of 11,557 (head SHA, workflow, test) groups (0.54%) and remove 46 of 4,214 relaxed labels (1.09%).

Binding. A label is more useful when it names a file. We bind each `test_id` to a test source file inside a clone pinned to a recorded commit, never to a moving branch head, so the result does not depend on when a machine last fetched. A unique match on the fully qualified class name has confidence 1.0; a match on the bare class name has confidence 0.5.

III. Storage and Schema

BR-Bench consists of four Apache Parquet tables, documented by a datasheet in the form proposed by Gebru et al. [9]. Table I lists their main fields. The `instances` table has one row per workflow run. The `outcomes` table has one row per (run, test, split) triple, over 4,384 distinct run and test pairs. The `cochange` table holds pairs of files edited in the same commit, with support, lift and confidence in both directions, and `failure_messages` holds the failure text recorded for a labelled test. Every count in this section can be read directly from the release files.

Releases. Release v0.1 is published on Zenodo [10]. Release v0.2 is built and validated against its schema locally; it is not yet deposited. It keeps the four tables and their row counts and adds fields. In `instances` these are `base_run_id` (filled for 7,061 runs), `base_status`, `base_run_distance` (filled for 6,025 runs), a typed `run_started_at`, and the changeset fields, which are null for the 17,364 runs without a recorded diff. In `outcomes` they are `test_file` (4,229 of 4,384 pairs, 96.5%), the binding status and confidence, `n_job_rows` (above 1 for 690 pairs, i.e., tests that ran in several matrix legs), `duration_s` (2,505 pairs, 57.1%) and the failure-message hash (2,111 pairs, 48.2%).

TABLE I
Main Fields of the BR-Bench Tables (Release v0.2)

Field	Content
instances *(165,349 rows; one per workflow run)*	
`instance_id`	key; SHA-256 of repository, head SHA and run id
`repo`, `language`	`owner/name`; Java or Python
`head_sha`, `base_sha`	head commit; target-branch commit
`run_started_at`	UTC start time, typed timestamp
`base_run_id`	resolved base run
`base_status`	one of the five statuses, or `not_attempted`
`base_run_distance`	commits between base run and B (at most 10)
changeset fields	`n_files_changed`, `n_lines_changed`, `touches_*`, `is_docs_only`, `changed_files`; null without a diff
`author_login`	keyed hash, 16 hex characters
outcomes *(12,766 rows; one per run, test and split)*	
`run_id`, `test_id`	run key; canonical test identifier
`split`	`all`, `relaxed` or `strict`
`test_file`, `binding_*`	bound source path; binding status and confidence
`status_head`	head verdict (fail or error)
`n_job_rows`	matrix legs merged into the row
`duration_s`	test duration in seconds
`failure_message_hash`	SHA-256 of the failure text
cochange *(175,204 rows)*: `file_a`, `file_b`, `support`, `lift`, `conf_*`	
failure_messages *(3,490 rows)*: `run_id`, `job_id`, `test_id`, `failure_message`	

TABLE II
Attrition of Repositories and Runs Through the Pipeline

Stage	Count	% of prev.	% of first
Repositories			
Exported from SEART	3,671	–	100.00%
$\geq$ 100 runs in 90 days	2,646	72.08%	72.08%
Has a test workflow	2,333	88.17%	63.55%
In the sampling frame	300	12.86%	8.17%
Reached by the harvester	76	25.33%	2.07%
Has a failed run	69	90.79%	1.88%
Has a strict instance	37	53.62%	1.01%
Workflow runs			
Discovered	165,349	–	100.00%
Failed	12,581	7.61%	7.61%
Failed-job log on disk	6,586	52.35%	3.98%
At least one parsed failing test	1,774	26.94%	1.07%
Base run resolved	1,348	75.99%	0.82%
Base failure set known	897	66.54%	0.54%
At least one strict label	762	84.95%	0.46%

Privacy. We replace each author login with an HMAC-SHA256 hash under a key the operator holds, truncated to 16 hex characters; the release has 1,791 distinct hashed authors. A secret scan runs before every release. For v0.1 it found no blocking secret, and the authors checked each finding it flagged for review by hand. The findings flagged in v0.2 still await that check, which is one reason v0.2 is not yet deposited.

IV. Dataset Overview and Quality

Attrition. Table II follows repositories and runs through the pipeline. The largest losses occur at the logs. Of the 12,581 failed runs, 6,586 (52.35%) have a failed-job log on disk, and only 1,774 of these contain a failing test that the parsers

TABLE III
SPLITS BY SIZE AND LANGUAGE

Split	Inst.	Labels	Tests	Java inst.	Python inst.
`all`	897	4,384	2,500	767 (85.5%)	130 (14.5%)
`relaxed`	770	4,214	2,484	646 (83.9%)	124 (16.1%)
`strict`	762	4,168	2,466	638 (83.7%)	124 (16.3%)

TABLE IV
BINDING OF TEST IDENTIFIERS AND CLASSES OF FAILURE

Measure	Count	Share
Binding, over 5,985 (repository, `test_id`) pairs		
Fully qualified class name, $c = 1.0$	3,820	63.83%
Bare class name only, $c = 0.5$	1,823	30.46%
Bound in total	**5,643**	**94.29%**
Not in the pinned tree	178	2.97%
Several matching files	164	2.74%
Failure class, over 4,168 strict labels		
Code-level assertion	729	17.49%
Timeout	112	2.69%
Environment (GPU, network, service)	89	2.14%
Unknown	3,238	77.69%

recognise. A base run is found for 1,348 of them. For 897 the base failure set is known, either because the base run was `exact_green` or because its log was parsed, and 762 keep at least one strict label.

Composition. Table III gives the size of each split. Java dominates. It accounts for 638 of the 762 strict instances (83.73%), 3,855 of the 4,168 strict labels (92.49%) and 35 of the 37 repositories; Python contributes 124 instances and 313 labels from 2 repositories.

Binding and failure classes. Table IV reports binding over 5,985 distinct (repository, `test_id`) pairs. Of these, 5,643 (94.29%) bind to a file, 3,820 by fully qualified class name and 1,823 by bare class name. The other 342 are either absent from the pinned tree (178) or match several files (164). We also sort the failure message of each strict label into classes with an ordered list of regular expressions. The result is a lower bound on environment failures, because a label without a recorded message counts as unknown. Of the 3,238 unknown labels, 2,231 have no recorded message at all.

V. HOW THE DATA HAS BEEN USED: RQ1

We use the data to ask one question. *RQ1: how well do cheap proxies for change impact find the fault-revealing test files of a change?*

Methods. Each method returns at most k test files for an instance. Co-change (all) returns the files edited together with the changed files most often, counted over the 365 days of commits before the run started (support at least 2). Co-change (test) keeps only conventional test files (paths under `src/test/`, names such as `*Test.java` or `test_*.py`). The changeset baseline returns the changed files themselves. Failure history returns the k test files with the most strict labels in runs that started earlier, each (test, run) counted once.

Leakage. An earlier version of this measurement saw the future. A static co-change table, mined at the end of the harvest, contained commits made after the run in all 713 audited instances, and in 640 of them such a commit touched a changed file. The failure-history baseline counted labels from all three splits (224,758 rows for 78,399 distinct labels). With

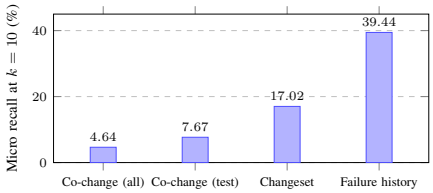


Fig. 2. Micro recall at $k = 10$ of four proxies on the same 576 strict instances (1,316 fault-revealing test files).

trailing windows, lookups in both directions of a file pair, and one count per strict label, the audit finds no violation (0 of 713 co-change instances, 0 of 762 failure-history instances).

Results. Of the 734 strict instances with at least one bound label, 576 have a non-empty co-change set (462 Java, 114 Python), and we score every method on exactly these, with 1,316 fault-revealing test files. Fig. 2 shows micro recall at $k = 10$. Co-change (all) finds 61 of the 1,316 files (4.64%) and co-change (test) 101 (7.67%). The changeset baseline finds 224 (17.02%) and failure history 519 (39.44%). The order does not change when environment and timeout labels are removed. Co-change is a poor proxy for the tests a change breaks. Failure history needs no analysis of the code at all, and on these data it is the baseline to beat.

VI. ORIGINALITY AND RELATED DATASETS

CI datasets. The closest public dataset is RTPTorrent [11]. Its unit is a Travis CI build job of a Java project and its target use is test prioritisation, so it does not tie a diff to the tests that the diff broke. GHALogs [12] holds far more GitHub Actions logs than we do, but stores them unprocessed; a user still has to parse tests, find base runs and remove flaky tests. BR-BENCH performs those three steps and publishes the result.

Selection and impact tools. Ekstazi [3] tracks file dependencies at run time, STARTS [4] computes class dependencies statically, and Chianti [13] splits a change into atomic edits and finds the tests each one affects. These tools aim at safety (no affected test is skipped); BR-BENCH records which tests did fail. The two views fit together, since our labels could measure how much larger a safe selection is than the set of tests that matter.

Industrial studies. The predictive selection system of Machalica et al. [1] and the Google study of Memon et al. [2] rest on internal monorepos whose data are not released.

VII. LIMITATIONS AND THREATS

Gates not met. Before collection we set two targets, Gate 1 (at least 5,000 strict labels) and Gate 1.5 (at least 70% binding). The release reaches 4,168 labels, 83.36% of the target, and 762 instances, 15.24% of the same figure read as instances. Gate 1.5 is met on combined binding (94.29%) and not met at full confidence (63.83%).

Parser precision. The parsers score 46 of 46 on the fixture corpus and 30 of 30 on holdout v1, but both sets guided development, so these are fit figures, not held-out precision. An independent holdout (v5) exists and is not scored, because no blind human annotation was completed.

Observational labels. We re-ran no test. A label says that a test failed in CI at the head and not at the base; it does not

prove that the change caused the failure. Environment failures and timeouts remain in the strict split.

Log expiry. GitHub Actions deletes logs after 90 days. At the last capture (29 August 2026), 4,458 of the 13,422 failed runs in the harvest corpus (33.21%) were already older than that, so a fresh harvest cannot rebuild the dataset. Rebuilding the labels from logs needs our raw log store, which the Zenodo bundle does not contain.

Skew. 35 of the 37 strict repositories are Java projects, and a small number of active repositories contribute a large share of the instances.

VIII. Future Research Questions

The data suggest three directions. (1) *Learned selection:* train rankers on diffs and failure history and compare them with failure history alone under a chronological split. (2) *Re-execution:* re-run head and base in containers to separate regressions from environment failures. (3) *Coverage:* add further languages and test runners. The repository also contains a ranking benchmark and a repair benchmark built on BR-Bench; this paper does not report them.

IX. Availability and Reproducibility

The code is released under the MIT licence and the tables under CC BY 4.0; the mined repositories keep their own licences. Release v0.1, with `instances`, `outcomes`, `cochange` and `failure_messages`, is on Zenodo [10] (DOI 10.5281/zenodo.23250262), and the code is archived there as well [14] (DOI 10.5281/zenodo.23250690). Release v0.2 is built and validated locally and is not yet deposited. Every number outside Section III regenerates offline with `make tables` (`BR_OFFLINE=1`); the counts in Section III are read from the release files. The artefact's own test suite has 1,041 tests; under `make test`, 1,039 pass and 2 are skipped.

X. Concluding Remarks

BR-Bench links pull-request changes to the tests they broke in real CI runs, after removing failures inherited from the base and failures that flip on one commit. Two questions stay open. How many strict labels would survive an isolated re-run? And does any signal drawn from the code add to failure history once both are measured on the same labels? We hope the data make both questions testable.

Acknowledgment

Generative AI assistants (Google Antigravity and Anthropic Claude) helped draft and edit parts of the text, check references and develop the artefact's software. The authors reviewed, rewrote and checked all content. No AI system takes part in data collection, labelling or the computation of any reported number; every number regenerates from the artefact through `make tables` and the release files.

References

[1] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, "Predictive test selection," in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2019, pp. 91–100.

[2] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming Google-scale continuous testing," in *Proc. 39th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.

[3] M. Gligoric, L. Eloussi, and D. Marinov, "Practical regression test selection with dynamic file dependencies," in *Proc. 2015 Int. Symp. Softw. Test. Anal. (ISSTA)*. ACM, 2015, pp. 211–222.

[4] O. Legunsen, A. Shi, and D. Marinov, "STARTS: STAtic regression test selection," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Tool Demos*. IEEE, 2017, pp. 949–954.

[5] M. Borg, K. Wnuk, B. Regnell, and P. Runeson, "Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context," *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 675–700, 2017.

[6] Y. Huang, J. Jiang, X. Luo, X. Chen, Z. Zheng, N. Jia, and G. Huang, "Change-patterns mapping: A boosting way for change impact analysis," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2376–2398, 2022.

[7] O. Dabic, E. Aghajani, and G. Bavota, "Sampling projects in GitHub for MSR studies," in *Proc. 18th Int. Conf. Mining Softw. Repos. (MSR)*. IEEE, 2021, pp. 560–564.

[8] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*. ACM, 2018, pp. 433–444.

[9] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[10] Deepanshu, S. Singh, P. Vadhavkar, and Yoga Raja C A, "BR-Bench: Execution-grounded change-to-failing-test data from GitHub actions (release v0.1)," https://doi.org/10.5281/zenodo.23250262, 2026.

[11] T. Mattis, P. Rein, F. Dürsch, and R. Hirschfeld, "RTPTorrent: An open-source dataset for evaluating regression test prioritization," in *Proc. 17th Int. Conf. Mining Softw. Repos. (MSR)*. ACM, 2020, pp. 385–396.

[12] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, "GHALogs: Large-scale dataset of GitHub actions runs," in *Proc. 22nd Int. Conf. Mining Softw. Repos. (MSR), Data and Tool Showcase Track*. IEEE, 2025, pp. 669–673.

[13] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley, "Chianti: A tool for change impact analysis of Java programs," in *Proc. 19th ACM SIGPLAN Conf. Object-Oriented Program., Syst., Lang., Appl. (OOPSLA)*. ACM, 2004, pp. 432–448.

[14] Deepanshu, S. Singh, P. Vadhavkar, and Yoga Raja C A, "Blastradius and BR-Bench: Execution-grounded change impact data from GitHub actions," https://doi.org/10.5281/zenodo.23250690, 2026.